

AhamX Bodhi

Where I Becomes Infinite

Will start @ 11.10 am

Prompt Engineering Principles

Module 2.3: Prompt Engineering | Prof. Sashikumar Ganesan |
IISc Bengaluru



Session Overview

What You Will Learn

- What prompt engineering is and why it matters
- Core principles that make prompts effective
- The anatomy of a well-structured prompt
- How LLMs respond to different prompt styles

Session Details

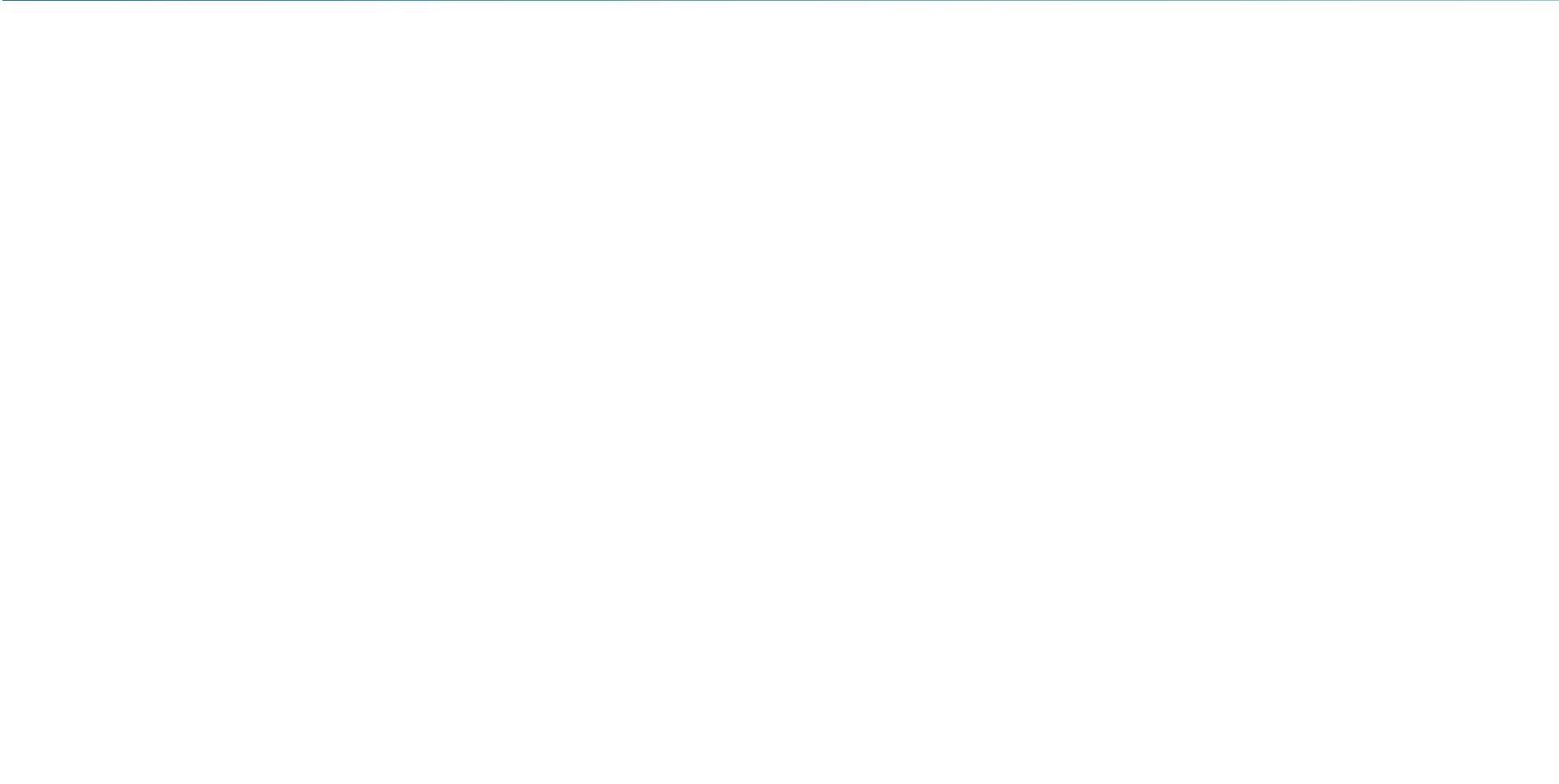
- **Duration:** 8 minutes
- **Bloom's Level:** B2 — Understand
- **Difficulty:** L1 — Beginner
- **Module:** 2.3 Prompt Engineering
- **Prereq:** What is Generative AI?



Learning Objectives

By the End of This Session You Will Be Able To

- **Define** prompt engineering and its role in the GenAI stack
- **Explain** why the same model produces different outputs for different prompts
- **Identify** the key components of an effective prompt
- **Apply** core principles to improve your own prompts
- **Distinguish** between good and poor prompting practices





Prompt Engineering: A Definition

Core Definition

Prompt engineering is the practice of designing, refining, and optimizing natural language inputs to guide a large language model toward producing accurate, relevant, and useful outputs for a given task.

Unlike traditional programming, prompt engineering does not require modifying model weights. You shape behaviour entirely through the text you provide.

Why It Matters

- LLMs are general-purpose: prompts specialize them for your task
- A well-engineered prompt can replace hundreds of lines of rule-based code
- Prompt quality directly determines output quality in production systems



Where Prompt Engineering Fits

Five-Level GenAI Framework

- **Level 1:** Foundation Models — pre-trained LLMs (GPT-4, Claude, Llama)
- **Level 2 (You Are Here):** Model Adaptation — prompt engineering, RAG, fine-tuning
- **Level 3:** Applications — chatbots, agents, reasoning systems
- **Level 4:** Integration — embedding AI in products and workflows
- **Level 5:** Innovation — novel systems and research

Key Insight

Prompt engineering is the **lowest-cost, highest-leverage** way to customize an LLM. No GPU, no training data, no model changes required.



The Anatomy of a Prompt

Four Core Components

- **Role** — Assigns a persona to the model. Example: "You are a senior data scientist."
- **Context** — Background information the model needs. Example: "The following table shows Q4 2024 revenue by region."
- **Instruction** — The task directive: what to do. Example: "Identify the top two growth trends and explain them."
- **Output Indicator** — Desired format or structure. Example: "Return exactly two bullet points, each under 30 words."

Not every prompt needs all four components. Knowing which to include and how to phrase them precisely is the craft of prompt engineering.



Principle 1: Be Clear and Specific

Clarity Principle

Ambiguous language forces the model to guess your intent. Vague prompts produce vague outputs; precise prompts produce precise outputs.

In Practice

- **Vague:** "Tell me about Python."
- **Clear:** "Explain Python list comprehensions to a junior developer, with a concrete example transforming a list of integers."
- Name the audience, the format, the scope, and the desired depth
- Use active verbs: summarize, classify, extract, translate, compare

Draft



Principle 2: Provide Sufficient Context

Context Principle

LLMs have no memory between conversations. They only know what is in the current prompt. Missing context is the leading cause of irrelevant or incorrect responses.

Types of Context to Include

- **Role context:** "You are a senior security engineer reviewing code."
- **Domain context:** "This is a medical imaging classification task."
- **Constraint context:** "Answer only using information in the document below."
- **Format context:** "Output must be valid JSON, no additional text."



Principle 3: Specify the Output Format

Format Principle

Without format guidance, LLMs default to verbose paragraph responses. Explicit format instructions constrain and direct their output to match pipeline requirements.

Common Format Directives

- "Respond with a numbered list of exactly five items."
- "Return only a valid JSON object; no explanation."
- "Write in the style of a technical specification document."
- "Limit your response to 100 words."
- "Use Markdown with H2 headers for each section."

Markdown files - MD-



Principle 4: Assign a Role or Persona

Role Principle

Assigning the model a role activates domain-relevant knowledge and adjusts tone. The model primes itself with vocabulary and reasoning patterns appropriate to the assigned persona.

Role Assignment Examples

- "Act as an experienced DevOps engineer." — infrastructure-focused answers
- "You are a patient tutor explaining to a 12-year-old." — simplified language
- "You are a skeptical peer reviewer." — critical, questioning responses
- "Act as a Python code reviewer following PEP 8." — style-specific feedback

Principle 5: Iterate and Refine

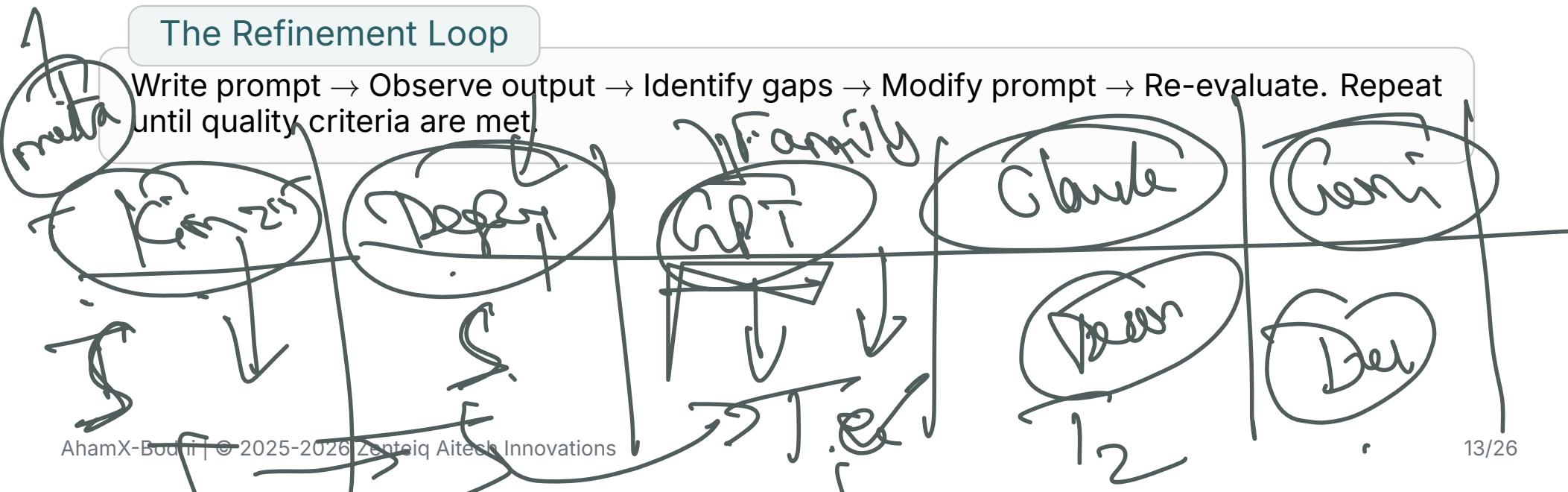


Iteration Principle

Prompt engineering is empirical. The first prompt rarely produces the ideal output. Systematic refinement — adjusting wording, adding constraints, restructuring components — is the core workflow of a prompt engineer.

The Refinement Loop

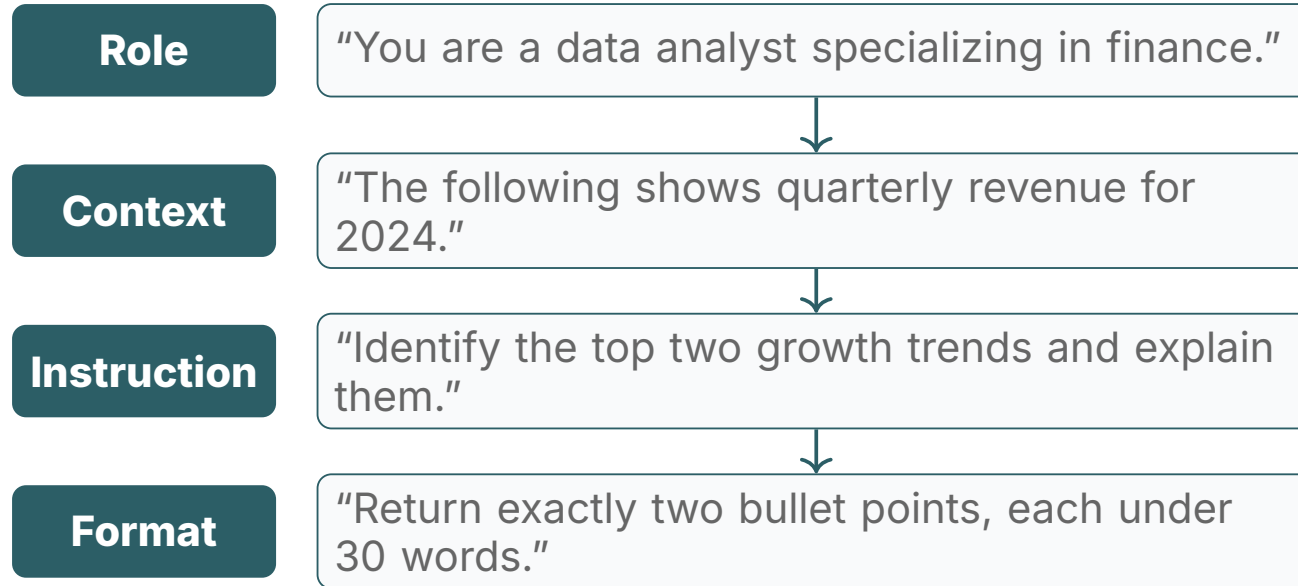
Write prompt → Observe output → Identify gaps → Modify prompt → Re-evaluate. Repeat until quality criteria are met.







Prompt Structure: Visual Overview



Each layer narrows the model's solution space, guiding it toward exactly the response you need.



Common Prompt Engineering Pitfalls

What NOT To Do

- **Underspecification:** "Write me something about AI." — no scope, format, or audience
- **Conflicting instructions:** "Be concise. Also include comprehensive examples."
- **Assumed knowledge:** Referencing a prior conversation the model cannot see
- **No output constraint:** Open-ended questions in automated pipelines
- **Prompt stuffing:** Packing 10 different tasks into a single prompt



Positive vs. Negative Instructions

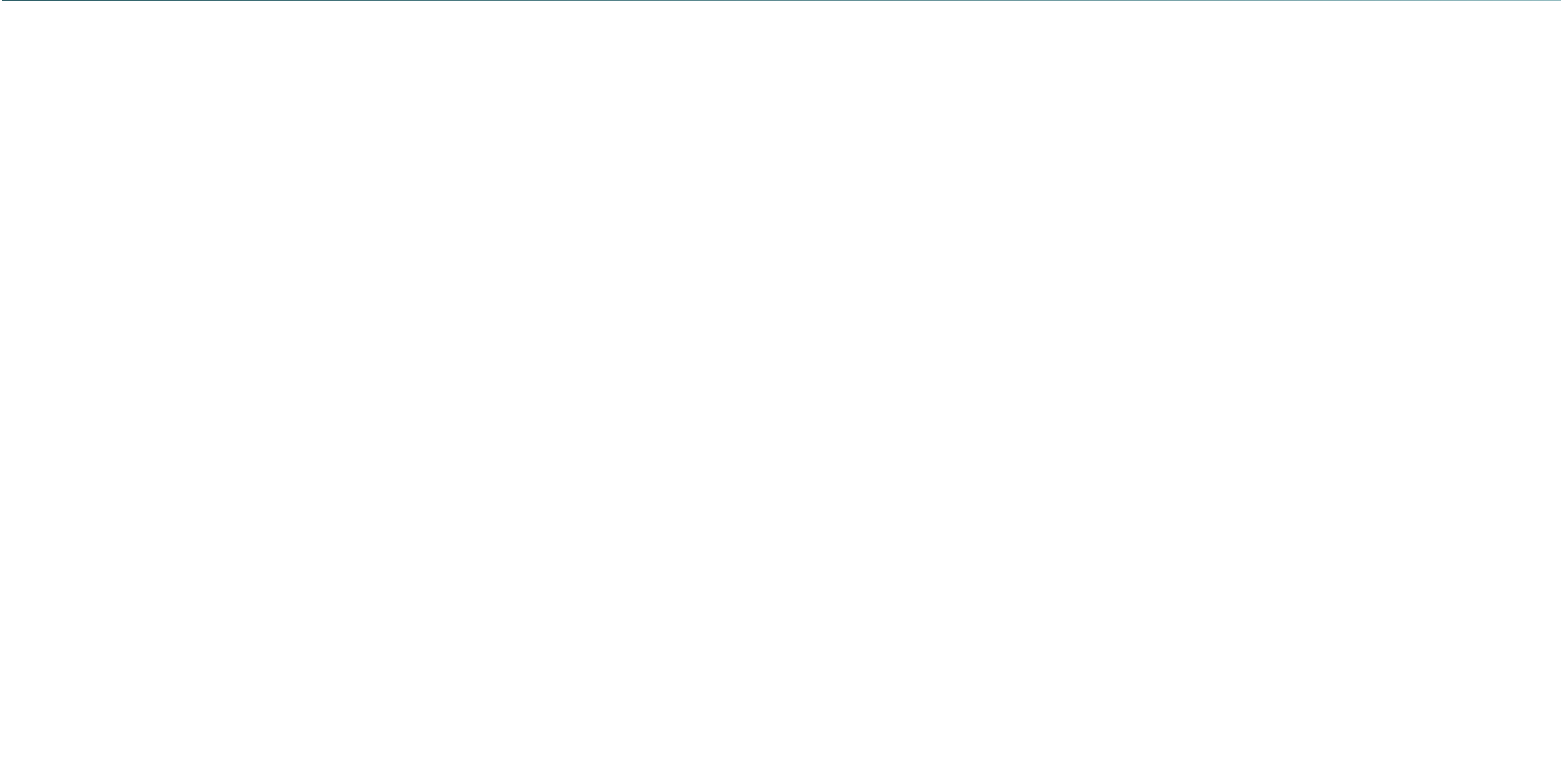
Prefer Positive Instructions

Tell the model what TO DO rather than what NOT to do.

- **Avoid:** "Don't be verbose."
- **Better:** "Use at most two sentences per point."
- **Avoid:** "Don't use jargon."
- **Better:** "Use plain language for a non-technical reader."

When Negatives Are Needed

- Safety guardrails: "Never include personal information."
- Scope restrictions: "Do not answer outside the document."
- Format exclusions: "Do not include any code blocks."





Application: Automated Customer Support

Production Deployment at Scale

Companies such as Intercom, Zendesk, and Salesforce embed LLMs in customer support pipelines. A carefully engineered prompt defines the agent's persona, knowledge scope, tone, and escalation rules. The same base model serves thousands of brands, each shaped entirely through prompt design.

Key Prompt Engineering Decisions

- System prompt sets the brand persona and topic boundaries
- Context window includes relevant product documentation
- Output format ensures structured, actionable replies
- Fallback instructions handle out-of-scope queries gracefully



Application: Code Generation Assistants

GitHub Copilot, Cursor, Amazon CodeWhisperer

Code generation tools use engineered prompts combining file context, cursor position, language metadata, and user intent. The prompt is constructed programmatically from your IDE state and sent to an LLM to generate completions.

Prompt Components in Code Assistants

- **Language tag:** "Python 3.11, PEP 8 style"
- **File context:** Surrounding functions, imports, docstrings
- **Instruction:** "Complete this function to sort by date descending"
- **Format constraint:** Return only valid Python, no explanation



Application: Retrieval-Augmented Generation

Prompt Engineering Anchors RAG Quality

In RAG systems (ChatGPT Plugins, Perplexity, Bing Chat), the LLM receives a dynamically constructed prompt containing retrieved documents plus the user query. The quality of the prompt template determines whether the model grounds its answer in retrieved context or hallucinates.

RAG Prompt Engineering Pattern

- Instruction: "Answer only using the provided documents."
- Context: Top- k retrieved chunks injected at runtime
- Fallback: "If the answer is not in the documents, say so."



Application: Agentic AI Systems

Prompts as Agent Configuration

Autonomous AI agents (LangChain agents, AutoGPT, Claude Computer Use) are orchestrated almost entirely through prompt engineering. The system prompt defines the agent's goal, available tools, reasoning protocol, and output format.

Safety-Critical in Production

In agentic systems, prompt engineering is a **safety-critical** activity. Ambiguous instructions can cause agents to take unintended actions with real-world consequences: file deletion, API calls, financial transactions.



Key Takeaways



Core Principles to Remember

- Prompt engineering is the practice of crafting inputs to control LLM behaviour
- Every prompt has four potential components: role, context, instruction, output format
- Clarity, context, format, role, and iteration are the five core principles
- Positive instructions outperform negative instructions for reliability
- Prompt engineering powers production systems at massive scale with no model training



What Comes Next in This Module

Module 2.3 Progression

- **Next:** Zero-Shot Prompting — asking without examples
- **Then:** One-Shot and Few-Shot Prompting — guiding with examples
- **Then:** Chain-of-Thought (CoT) — reasoning step by step
- **Then:** Templates, Chaining, System Prompts, Output Formatting

Practice Now

- Try the same task with a vague vs. precise prompt in ChatGPT or Claude
- Observe how adding role and format specifications changes output quality

AhamX Bodhi

Where I Becomes Infinite